

DEVELOPMENT OF A SIMPLE HTTP SERVER USING PYTHON SOCKET PROGRAMMING

Submitted To:

Faculty Name: Abu Bakar Fahad

Department of Computer Science & Engineering

United International University

Submitted By:

Name : Sourav Saha

Student ID : 0112330638

Section : B

Course Information:

Course Title : Computer Networks Laboratory

Course Code: CSE 3712

1. Purpose of the Assignment

The purpose of this assignment is to design and implement a simple HTTP server using Python socket programming. The server should accept browser requests and display HTML pages based on user-entered URLs. This assignment provides practical experience with networking concepts and web communication.

2. Introduction

Modern web applications work through communication between browsers and servers. Whenever a user enters a website address, the browser sends a request and waits for the server to return the requested content.

Socket programming allows this communication process to happen by creating a connection between two endpoints. One endpoint acts as the client and another acts as the server.

In this assignment, Python sockets were used to simulate this communication process locally.

3. Concepts Used

HTTP Server

An HTTP server receives requests and returns responses through HTTP protocol.

Examples:

Successful Response:

HTTP/1.0 200 OK

Invalid Request:

HTTP/1.0 404 NOT FOUND

Socket Communication

Socket programming allows:

- Creating network communication
- Accepting requests
- Sending responses
- Managing browser connections

4. Tools and Environment

Development Environment:

- Visual Studio Code

Programming Language:

- Python

Browser Used:

- Google Chrome

Module:

- Socket

5. Project Structure

Files used in this assignment:

server.py

index.html

about.html

contact.html

6. Working Procedure

The project was completed using the following procedure:

Step 1:

Create a server socket.

Step 2:

Assign localhost and port 8080.

Step 3:

Start listening for browser requests.

Step 4:

Read the requested URL.

Step 5:

Match the request with HTML files.

Step 6:

Send HTTP response.

Step 7:

Display webpage.

Step 8:

Return 404 response for invalid URLs.

7. Implementation Summary

The server successfully handled different URL paths.

Examples:

localhost:8080

→ Loads index page

localhost:8080/about.html

→ Loads about page

localhost:8080/contact.html

→ Loads contact page

localhost:8080/abc.html

→ Returns NOT FOUND

8. Source Code

server.py

```
from socket import *
host = "127.0.0.1"
port = 8080
http_server = socket(AF_INET, SOCK_STREAM)
http_server.setsockopt(
    SOL_SOCKET,
    SO_REUSEADDR,
    1
)
http_server.bind(
    (host, port)
)
http_server.listen(10)
print("Server Started")
while True:
    client, address = http_server.accept()
    request = client.recv(
        4096
    ).decode()
    try:
        request_line = request.split("\n")[0]
        url = request_line.split()[1]
        if url == "/":
            page = "index.html"
        else:
            page = url.replace(
                "/",
                ""
            )
    try:
        file = open(
            page,
```

```

"r",
    encoding="utf-8"
)
html = file.read()
file.close()
response = (
    "HTTP/1.0 200 OK\r\n"
    "Content-Type:text/html\r\n"
    "\r\n"
    + html
)
except:
    response = (
        "HTTP/1.0 404 NOT FOUND\r\n"
        "\r\n"
        "<h1>NOT FOUND</h1>"
    )
client.sendall(
    response.encode()
)
except:
    pass
client.close()
index.html
about.html
contact.html

```

index.html

```

<!doctype html>
<html>
<head>
<title>Home</title>
<style>
body{
    background: #eef7ff;
    font-family: Arial;
    text-align: center;
    padding-top: 100px;
}
.box{
    background: white;
    width: 500px;
    margin: auto;
}

```

```
padding: 30px;
    border-radius: 15px;
}
</style>
</head>
<body>
    <div class="box">
        <h1>Python HTTP Server</h1>
        <p>This page is served using socket programming.</p>
    </div>
</body>
</html>
```

about.html

```
<!doctype html>
<html>
<head>
    <title>About</title>
    <style>
        body{
            font-family: Verdana;
            background: #fff9ec;
            padding: 80px;
        }
    </style>
</head>
<body>
    <h1>About This Server</h1>
    <p>
        This project demonstrates how a browser communicates with a local HTTP
        server using Python socket programming.
    </p>
</body>
</html>
```

Contact.html

```
<!doctype html>
<html>
<head>
    <title>Contact</title>
```

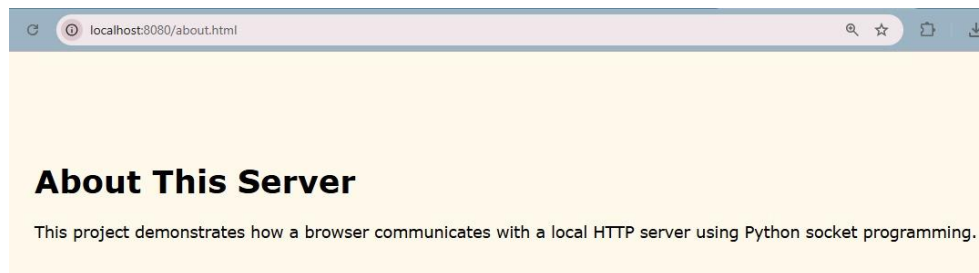
```
<style>
  body{
    background: #f4fff5;
    font-family: Tahoma;
    text-align: center;
    margin-top: 120px;
  }
</style>
</head>
<body>
  <h1>Contact Page</h1>
  <p>Email: sourav@gmail.com</p>
  <p>Phone: +8801790306890</p>
</body>
</html>
```

9. Experimental Results

Result 1:

Python HTTP Server.

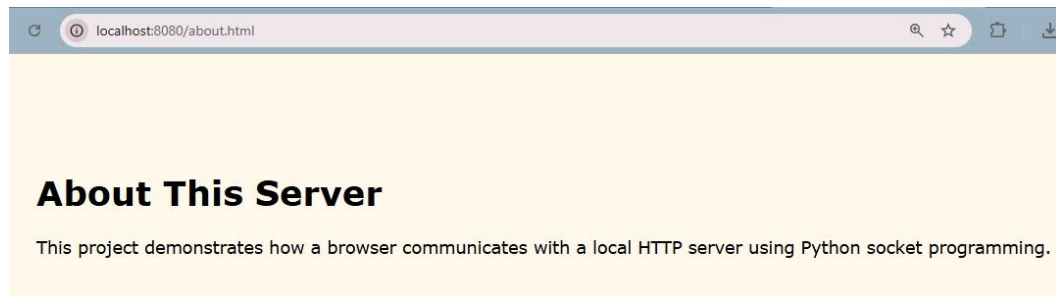
This page is served using socket programming.



Result 2:

About this server.

This project demonstrates how a browser communicates with a local HTTP server using Python socket programming.



Result 3:

Contact Page

Email: sourav@gmail.com

Phone: +8801790306890



Result 4:

NOT FOUND.



10. Learning Outcome

This assignment improved understanding of:

- Client-server architecture
- Socket programming
- HTTP communication

- URL-based webpage rendering
- Error handling

11. Conclusion

This assignment successfully demonstrated the implementation of a local HTTP server using Python socket programming. The server responded correctly to browser requests and delivered HTML pages based on user input.

This experiment provided practical knowledge about networking and the internal operation of web servers.